

Socket в Python

Socket – базовый модуль в Python.

Сокеты позволяют нам отправлять информацию между компьютерами наборы байтов. Нам необходимо создать серверное приложение и клиентское – достаточно два разных файла .py в одном проекте.

Создаём **серверное** приложение:

Перед работой обязателен импорт модуля.

```
import socket
```

После нам необходимо создать экземпляр класса Socket.

```
sock = socket.socket()
```

Теперь настраиваем параметры, указывая ip и порт:

```
sock.bind(('localhost', 9090))
```

После мы готовы к работе, пропишем код для прослушивания подключения, благодаря методу listen(). Listen() может принимать аргумент – кол-во пользователей в очереди. Если listen(1), то второй клиент в очереди будет отвергнут.

```
sock.listen()
```

Для принятия подключения – используем метод accept(), он возвращает нам кортеж из 2 элементов – новый экземпляр класса socket для работы с данным подключением и информацию о подключении (ip).

```
conn, addr = sock.accept()
```

Для получения сообщений используется метод recv() – в качестве аргумента указывается количество байтов принимаемого сообщения, обычно recv(1024).

Для отправки сообщений метод send().

Но, мы уже ранее отметили, что все socket работает с байтовым форматом, поэтому будем использовать методы для кодирования и декодирования наших сообщений – encode("utf-8") и decode("utf-8").

Создадим простой метод для в бесконечном цикле для отправки и принятия сообщений. Если получаемые данные будут "exit", то цикл заканчивается и клиенту отправляется сообщение о выходе.

```
while True:
    data = conn.recv(1024).decode('utf-8')
    if data == 'exit':
        conn.send('exit'.encode('utf-8'))
        break
    print(f"Получено сообщение: {data}")
    conn.send(input("Введите сообщение: ").encode('utf-8'))
```

После, нам нужно закрыть соединение, для этого нужно использовать метод close().

```
conn.close()
```

Создаем **клиентское** приложение:

Тут все еще проще. Импортируем модуль socket и создаем экземпляр класса socket.

```
import socket

sock = socket.socket()
```

Теперь нам нужно подключиться к серверу через метод connect – в качестве аргументов нужно указать ip-адрес и порт подключения.

```
sock.connect(('localhost', 9090))
```

Отлично, мы подключились к серверу.

Теперь мы можем написать код для получения и отправки сообщений. Сделаем бесконечный цикл, в нем пропишем код для отправки и получения сообщений. Если полученное сообщение “exit” – то цикл прерывается.

```
while True:
    sock.send(input('Отправить сообщение: ').encode('utf-8'))
    data = sock.recv(1024).decode('utf-8')
    print(f'Получено сообщение: {data}')
    if data == 'exit':
        print('Отключение')
        break
```

После окончания цикла сделаем отключение от сервера:

```
sock.close()
```

Полный код серверного приложения:

```
import socket

sock = socket.socket()
sock.bind(('localhost', 9090))
sock.listen()
conn, addr = sock.accept()

while True:
    data = conn.recv(1024).decode('utf-8')
    if data == 'exit':
        conn.send('exit'.encode('utf-8'))
        break
    print(f"Получено сообщение: {data}")
    conn.send(input("Введите сообщение: ").encode('utf-8'))

conn.close()
```

Полный код клиентского приложения:

```
import socket

sock = socket.socket()
sock.connect(('localhost', 9090))

while True:
    sock.send(input('Отправить сообщение: ').encode('utf-8'))
    data = sock.recv(1024).decode('utf-8')
    print(f'Получено сообщение: {data}')
    if data == 'exit':
        print('Отключение')
        break
sock.close()
```

Заключение.

В этом руководстве мы научились работать с модулем socket, для передачи наборов байтов между компьютерами в одной сети.

Основные методы:

socket(), bind(), listen(), accept(), connect(), recv(), send(), close(), а также методы строк: encode(), decode().

Также с socket часто используют метод threading.